

CourseFlix Back-Of-The-Envelope Calculations

Context

CourseFlix is deployed per teacher.

That means each deployment serves one teacher, their assistants, their courses, and their students. This keeps the scale controlled and makes the infrastructure easier to reason about.

Assumptions

Item	Estimate
Teachers per deployment	1
Students per teacher	1,000
Daily active students	30% = 300 students
Assistants/teacher users online	5 users
Lessons watched per active student/day	3 lessons
AI/chat messages per active student/day	10 messages
Notification polling interval	30 seconds
Notification polling rate per user	2 requests/minute
Peak multiplier during exams	10x

AI / Chat Requests

Daily AI/chat messages:

$300 \text{ active students} * 10 \text{ messages/day} = 3,000 \text{ messages/day}$

Average requests per second:

$3,000 / 86,400 \text{ seconds} = \sim 0.035 \text{ requests/sec}$

Peak traffic during exams:

$0.035 * 10 = \sim 0.35 \text{ requests/sec}$

Conclusion

AI/chat traffic is manageable for one teacher deployment. The expensive part is not the number of chat requests, but the AI processing time, token usage, and document retrieval pipeline.

Notification Polling

Each active user sends:

$2 \text{ notification requests/minute}$

For 300 active students:

$300 * 2 = 600 \text{ requests/minute}$
 $600 / 60 = 10 \text{ requests/sec}$

For teacher and assistants:

$5 * 2 = 10 \text{ requests/minute}$
 $10 / 60 = \sim 0.17 \text{ requests/sec}$

Total notification polling:

$10 + 0.17 = \sim 10.17 \text{ requests/sec}$

Peak notification traffic:

$10.17 * 10 = \sim 102 \text{ requests/sec}$

Conclusion

Notification polling is the largest constant API load. For the MVP it is acceptable, but later it should move to WebSockets or Server-Sent Events, or use longer polling intervals and cached unread counts.

Video Views

Daily lesson views:

```
300 active students * 3 lessons/day = 900 lesson views/day
```

Average views per second:

```
900 / 86,400 = ~0.01 views/sec
```

Peak views:

```
0.01 * 10 = ~0.1 views/sec
```

Conclusion

The API should not serve video files directly. Videos should be stored in object storage and delivered through a CDN or static file service. The API only stores metadata, permissions, watch progress, and attendance events.

Message Storage

Assume average message size:

```
1 KB/message
```

Daily storage:

```
3,000 messages/day * 1 KB = 3 MB/day
```

Monthly storage:

$3 \text{ MB/day} * 30 = 90 \text{ MB/month}$

With metadata, indexes, citations, and overhead:

$\sim 300 \text{ MB to } 1 \text{ GB/month}$

Course Material Storage

Assume the teacher uploads:

20 PDFs/month
 $\text{average PDF size} = 10 \text{ MB}$

Raw file storage:

$20 * 10 \text{ MB} = 200 \text{ MB/month}$

Embeddings, extracted text, chunks, and metadata:

$\sim 50 \text{ MB to } 200 \text{ MB/month}$

Total course material storage:

$\sim 250 \text{ MB to } 400 \text{ MB/month}$

Monthly Storage Per Teacher Deployment

Area	Estimate
Chat/messages	300 MB to 1 GB
Course files + embeddings	250 MB to 400 MB
Logs + metadata	100 MB to 300 MB
Total	$\sim 650 \text{ MB to } 1.7 \text{ GB/month}$

Recommended Server Size

MVP / Demo Deployment

Good for demos, early testing, and a small teacher deployment.

Component	Recommendation
CPU	2 vCPU
RAM	4 GB
Disk	40-60 GB SSD
Database	PostgreSQL on same server or small managed DB
Redis	Same server
Chroma/vector DB	Same server
Worker	Same server, 1 worker process

This can handle the MVP because average traffic is low. The main risk is AI/document processing jobs competing with the API for memory and CPU.

Recommended Production Per Teacher

Better for a real teacher with around 1,000 students.

Component	Recommendation
CPU	4 vCPU
RAM	8 GB
Disk	80-120 GB SSD
Database	Managed PostgreSQL or separate DB container
Redis	Small managed Redis or same machine
Vector DB	Same server is acceptable at this scale
Worker	Separate process/container from API

Video files	Object storage + CDN
-------------	----------------------

This is the best balanced option for one teacher deployment. It gives enough room for API requests, polling, background AI jobs, PDF parsing, embeddings, and database indexes.

Safer Production / Exam Periods

Useful if many students are active at the same time during exams.

Component	Recommendation
CPU	4-8 vCPU
RAM	16 GB
Disk	160 GB SSD
Database	Managed PostgreSQL
Redis	Managed Redis
Worker	1-2 separate worker containers
Vector DB	Separate Chroma service/container
Video files	Object storage + CDN

This setup is safer when notification polling spikes, AI usage increases, or the teacher uploads many PDFs/videos.

Practical Recommendation

For CourseFlix, per teacher deployment:

```
4 vCPU
8 GB RAM
80-120 GB SSD
PostgreSQL
Redis
API process
Worker process
Chroma/vector DB
Object storage/CDN for videos
```

This is enough for the expected load:

```
~3,000 AI/chat messages/day
~10 requests/sec notification polling average
~100 requests/sec notification polling peak
~650 MB to 1.7 GB new storage/month
```

Scaling Decisions

Start simple:

- One API server.
- One background worker.
- PostgreSQL for relational data.
- Redis for queues/cache.
- Chroma for vectors.
- Object storage/CDN for videos.

Scale later when needed:

- Move notifications from polling to WebSockets or SSE.
- Cache unread notification counts.
- Move workers to separate machines/containers.
- Use managed PostgreSQL.
- Put Chroma on a separate service.

- Add horizontal API replicas behind a load balancer.